Elementor AI Page System — Complete Work Process

A repeatable system for building on-brand Elementor pages for **any** website, fast, using Claude. You point Claude at a site's exported Elementor kit; it learns that site's brand and builds new pages from a content doc that import cleanly and look like they always belonged.

> ***Designed to run with Claude.*** *Ideally Claude Code or Cowork (with file access and a Linux shell), because the workflow reads a kit folder, writes skill files, and generates Elementor JSON. A plain claude.ai Project can hold the instructions but cannot read the kit, write skills, or emit JSON — so use Code/Cowork.*

---

0. Team setup — get the toolkit running (~10 min)

A. Get the files

This entire `Elementor-AI-Page-System/` folder is the toolkit. To share it with a teammate: zip the whole folder (or put it in your team Git repo / shared drive) and send it. They download and unzip it anywhere on their computer. Inside:

- `docs/` — the process (playbook, SOP, content-brief template, publishing & QA checklist)
- `skills/` — the skills as folders (for **Claude Code**)
- `skills-zipped/` — the two portable skills pre-zipped (for **Claude Desktop** upload)
- `examples/` — real finished pages for reference

B-1. Use it in Claude Code

1. Install Claude Code and open a terminal.
2. Put the site's Elementor export in a project folder, e.g. `my-site/`.
3. Copy the two portable skill folders into the project's skills directory: `my-site/.claude/skills/elementor-kit-onboarding/` and `my-site/.claude/skills/full-output-enforcement/`. (Or copy them into `~/.claude/skills/` to make them available in every project.) Claude Code auto-discovers skills and reloads them live — no restart needed.
4. Copy `docs/Elementor-Site-Playbook.md` to the project root as `CLAUDE.md` — Claude Code loads it automatically as context.

5. Run `claude` in the project folder and say: **"Onboard this Elementor kit."**

B-2. Use it in Claude Desktop (Cowork)

1. Open Claude Desktop -> **Customize -> Skills -> "+" -> "Create skill"**, and upload the zips from `skills-zipped/` (start with `elementor-kit-onboarding.zip` and `full-output-enforcement.zip`). Toggle them on.
2. In a Cowork chat, **connect the folder** that holds the site's Elementor export.
3. Keep `docs/Elementor-Site-Playbook.md` and `docs/content-brief-template.md` in that folder for reference.
4. Say: **"Onboard this Elementor kit."**

C. Then (either tool)

Verify the generated skills -> hand over a filled `content-brief-template.md` -> say **"Build a page"** -> import the returned template into Elementor. Sections 3-5 have the detail.

> *The example `vitalair-*` skills (under `skills/example-site-vitalair/`) are included only to show what onboarding produces. They are **not** zipped for install and you do **not** add them to other sites — the onboarding step generates fresh `<site>-*` skills for each website.*

---

1. The core idea (read this first)

There are **two layers**, and keeping them separate is the whole trick:

- **Portable layer (build once, reuse forever):** the process + two site-agnostic skills. Never changes site to site.
- **Per-site layer (generated once per website):** the brand-specific skills that carry a single site's colors, fonts, voice, and templates.

You never re-teach the process. For each new site you run a one-time "onboarding" that *generates* that site's skills. After that, making pages is just:

content doc -> "build" -> import.

---

2. What's in this folder

```
Elementor-AI-Page-System/
■■■ README.md                          <- this file (the master process)
■■■ docs/
■   ■■■ Elementor-Site-Playbook.md     <- the portable 5-step process (use as CLAUDE.md)
■   ■■■ Page-Creation-SOP.md           <- human step-by-step SOP
■   ■■■ content-brief-template.md      <- fill-in brief you hand Claude per page
■   ■■■ Example-Kit-Analysis-VitalAir.md <- example of the analysis Claude produces
■   ■■■ Publishing-QA-Checklist.md     <- pre-publish QA gate (SEO, links, media, mobile)
■■■ skills/
■   ■■■ elementor-kit-onboarding/      <- PORTABLE: analyzes a kit, generates site skills
■   ■■■ full-output-enforcement/       <- PORTABLE: keeps Elementor JSON complete/valid
■   ■■■ example-site-vitalair/         <- EXAMPLE of what onboarding generates per site
■       ■■■ vitalair-design-read/
■       ■■■ vitalair-ui-design/
■       ■■■ vitalair-content-style/
■       ■■■ vitalair-page-builder/
■       ■■■ vitalair-page-audit/
■■■ skills-zipped/                     <- the two PORTABLE skills, pre-zipped for Claude Desktop upload
■■■ examples/                          <- real import-ready pages this system produced
    ■■■ VitalAir-Service-Areas.json
    ■■■ VitalAir-Norcross-GA-Image-Layout.json
```

---

3. The skills and how to use them

Skills are Markdown files (`SKILL.md`) with YAML frontmatter. Claude auto-invokes one when a request matches its `description` triggers. You can also name a skill explicitly ("use vitalair-page-builder"). Install them by placing each skill folder where Claude reads skills (Cowork: Settings -> Capabilities; Claude Code: your skills directory), or keep them in the site folder for reference.

Portable skills (copy to every site)

`elementor-kit-onboarding` — the engine.

- *Use when:* pointed at a new/unfamiliar Elementor export, or told "onboard this kit / set up skills for this site."
- *What it does:* mines the REAL design system from the widget styling (not `site-settings.json`, which is usually still Hello Elementor defaults), extracts the content voice, and writes the five per-site skills, then verifies them.
- *Output:* `<site>-design-read`, `<site>-ui-design`, `<site>-content-style`, `<site>-page-builder`, `<site>-page-audit`, plus a verification report.

`full-output-enforcement` — completeness guard.

- *Use when:* generating or editing any Elementor JSON.
- *Why:* Elementor exports are long and deeply nested; a truncated file fails to

import. This bans placeholders, enforces unique ids and required keys, and handles token-limit splits cleanly.

Per-site skills (generated by onboarding — VitalAir shown as the example)

`<site>-design-read` — the front door. Reads the brief, states a one-line "design read" (page kind + closest existing page to mirror), routes to the others. Use first on any page request.

`<site>-ui-design` — the visual system: palette + roles, type scale, button spec, the required **Section -> Content Container -> content** structure (padding lives only on the Content Container; children get none), **color-only button hover**, mobile column-stacking**, reusable header/footer/section IDs, and a review checklist. Style is applied **inline**, never via Elementor global slots.

`<site>-content-style` — the copy voice: tone, locality, formatting, reusable copy patterns pulled from real pages, CTA phrasings, do/don't list.

`<site>-page-builder` — the orchestrator. Runs the full pipeline: design read -> map content to the section anatomy -> write/polish copy -> style inline -> emit complete JSON -> audit -> export an importable template.

`<site>-page-audit` — the checker. Brand + Elementor-hygiene audit, WITH a "do NOT fix" list of intentional brand choices (so generic web-redesign instincts don't fight the brand — e.g. colored hero bands and alternating dark/light sections are correct here, not mistakes).

---

4. The end-to-end process (5 steps)

Step 1 — Export the Elementor kit

In WordPress: **Elementor -> Tools -> Export Kit** (include content + templates + site settings). Unzip into a fresh `<site>/` folder.

Step 2 — Onboard the site (generate its skills)

Put the portable skills + `docs/Elementor-Site-Playbook.md` (as `CLAUDE.md`) in the folder, open it with Claude, and say **"Onboard this Elementor kit."** Claude runs `elementor-kit-onboarding` and produces the five `<site>-*` skills + a report.

Step 3 — Double-check (verification gate)

Confirm the generated palette / font / type scale / button spec / voice match the live site. Confirm the section-structure and mobile rules are present. Fix the *skills*, not the output, if anything's off. Don't proceed until this passes.

Step 4 — Provide the content

Fill `docs/content-brief-template.md` (page type, location/topic, hero, sections, FAQ, CTA) or paste the content doc. Blanks get inferred from the closest existing page.

Step 5 — Build and import

Say **"Build a page from this brief."** Claude runs `<site>-page-builder` and returns an import-ready template. In WordPress: **Elementor -> Templates -> Import Templates **(the up-arrow icon) -> select the JSON ->** Insert**.

---

5. Elementor rules baked into this system (hard-won)

These are the lessons that separate "imports and looks right" from "broken." Every per-site `ui-design` and `page-audit` skill enforces them:

1. **Globals are usually fake.** `site-settings.json` often still holds Hello Elementor defaults (`#6EC1E4`, "Noto Sans Coptic"). Read styling from widgets; apply colors/fonts **inline**.
2. **Single-page import needs a template wrapper.** The kit `content/page/<id>.json` format throws **"Invalid template type."** For single-page import, wrap as:

```
{"version":"0.4","title":"<Page>","type":"page","content":[ ...elements... ],"page_settings":{"template":"default"}}
```

The `content/page/<id>.json` (+ manifest entry) format is only for **Import Kit**.

3. **Section structure:** every section = **Section (full-width, background, no padding) -> Content Container (holds the default padding, always) -> content**. Children and nested containers get **zero padding**; spacing comes from the Content Container's padding + gap.
4. **Button hover = color only.** No size/shape animation (`hover_animation` empty).
5. **Mobile:** multi-column rows must stack. Set child `width_tablet` (~48% for

3-col) and `width_mobile` (100%), plus per-breakpoint heading/body sizes and a smaller `padding_mobile` on the Content Container.

6. **Unique ids.** Every element needs a unique `id`; regenerate all ids when cloning a page so it imports as new.
7. **Complete JSON only.** No `// ...`, no collapsed repeated widgets, valid braces/escaping — truncated Elementor JSON won't import.
8. **Live widgets can't be embedded** (e.g. Google review sliders). Insert a labeled shortcode/placeholder and swap the real widget in after import.

5b. Elementor layout standards (the adjustments)

These structural rules are applied to every page the system builds; every per-site `ui-design`, `page-builder`, and `page-audit` skill enforces them.

1. Two-tier section structure. Every section is built as:

```
Section                (full-width outer container - background only, NO padding)
  ■■ Content Container   (the ONLY element with padding - always a default padding)
       ■■ content        (headings, text, buttons, nested columns/grids)
```

2. Padding discipline. Only the Content Container carries padding. Every child

- including nested containers, columns, and grids - has zero padding. All spacing comes from the Content Container's padding and the gap between elements.

3. Nested containers stay padding-free. Multi-column rows (feature banners, service grids, map + list) live inside the Content Container and never add their own padding.

4. Button behavior. Buttons change background color and text color on hover only - no grow, shrink, scale, or other size/shape animation.

Optional layout variant. A section can be split into two columns - content on one side and an empty column on the other, reserved for an image added manually in the editor (used on the Norcross hero and process sections).

5c. Content-to-live coverage (SEO, links, media, QA, posts)

The system takes content from AI draft all the way to a publish-ready page — not just layout. Every per-site skill enforces these; onboarding builds them into each

new site's skills.

- **SEO elements:** exactly one H1 with a clean heading hierarchy; meta title (< 60 chars), meta description (< 155 chars), and a lowercase-hyphenated slug carried from the brief; target keyword in the H1 and intro. (Elementor JSON doesn't store WordPress SEO meta, so title/description/slug are a publish-time handoff to set in Rank Math / Yoast.)
- **Internal linking:** required internal links with descriptive anchor text; no dead links; CTAs point to real destinations.
- **Image / media placement:** hero and feature images via the two-column or background pattern with dark overlay for legibility; alt text on every image; labeled placeholders for live widgets (review sliders, forms) to wire up after import.
- **Publishing & QA:** a final gate in `docs/Publishing-QA-Checklist.md` covering brand, structure, SEO, links, media, mobile, then the publish steps.
- **Posts as well as pages:** the same workflow builds blog posts — use the site's single-post template, set the brief's page type to "post," and handle post extras (featured image, category/tags, author, SEO) at publish.

6. Fastest build route: clone-and-swap

Prefer copying the closest existing page and swapping content over authoring from scratch: copy `content/page/<id>.json` -> replace text -> re-point location/topic -> regenerate all ids -> keep the on-brand styling -> audit. Author fresh only when no existing page is close.

---

7. Worked example (VitalAir)

This system was proven end-to-end on VitalAir (an Atlanta HVAC site):

- `docs/Example-Kit-Analysis-VitalAir.md` — the design system Claude extracted.
- `skills/example-site-vitalair/` — the five generated skills.
- `examples/*.json` — two real, import-ready pages built from content docs (a Service Areas page and a Norcross city hub with a two-column image-ready hero).

Use these as the reference for what a good onboarding + build looks like on the next site.

---

8. Quick start for the next site

1. Copy the **portable** skills (`elementor-kit-onboarding`, `full-output-enforcement`) + `docs/Elementor-Site-Playbook.md` (as `CLAUDE.md`) + `docs/content-brief-template.md` into the new `<site>/` kit folder.
2. Open with Claude -> "Onboard this Elementor kit."
3. Verify the generated skills.
4. Hand over a content brief -> "Build a page."
5. Import the returned template.

---

9. Known open extensions (not yet built)

- **Design-directions library:** a `design-directions/` shelf of brand-neutral, Elementor-aware aesthetics (e.g. a minimalist style) for greenfield/redesign sites, plus a "match existing kit vs. apply aesthetic" fork in the playbook.
- **One-click bundle:** packaging the portable set as a `.plugin` so setup is a single install.
- **Auto-fold finalized rules** (section structure, color-only hover, mobile stacking, two-column image-hero variant) directly into the templates the onboarding skill generates, so they're inherited automatically.